Computational Thinking & Write Clean functionality decomposed code

Trainer’s Name: Ms. Shanmuga Priya

Computational Thinking : Problem solving approach

main components are:

1. Decomposition
2. Pattern Recognition
3. Abstraction
4. Algorithm Design
5. Logic and Decision Making
6. Data Organization and Representation
7. Solution Evaluation and Optimization
8. Problem Generalization and Reusability

Computational Thinking

1. Decomposition

Breaking a large problem into smaller parts

Example : **Online Shopping Application**.

1. **User Registration**
2. **Login**
3. **Product Management**
4. **Shopping Cart**
5. **Payment Gateway**
6. **Order Management**
7. **Delivery Tracking**

2. Pattern Recognition

Identifying similarities or repeated patterns among problems

Name and phone number

Login Form
Registration Form
Feedback Form
Contact Form

3. Abstraction

Abstraction is the process of **focusing on the essential features while hiding unnecessary implementation details**. It allows users to interact with a system without knowing how it works internally.

3. Abstraction example

using Google Pay or PhonePe

- Enter the amount
- Enter the UPI PIN
- Click **Pay**

How the payment is encrypted

How banks communicate with app

How the money is transferred

How the transaction is verified

Only the required functionality is exposed to the user.

4. **Algorithm Design**

creating a clear, logical, and step-by-step procedure to solve a problem or complete a task efficiently.

- Solve the problem correctly.
- Have steps in the correct order.
- Produce the expected output.
- Be simple and efficient.

4. Algorithm Design

Start
↓
Read num1
↓
Read num2
↓
sum = num1 + num2
↓
Display sum
↓
Stop

4. Logic and Decision Making

Evaluating conditions and choosing the appropriate action based on the result.

When you insert your ATM card:

- If the PIN is correct → Continue to the next screen.
- Otherwise → Display **"Invalid PIN."**

When you request cash:

- If your account has sufficient balance → Dispense cash.
- Otherwise → Display **"Insufficient Balance."**

4. Data Organization and Representation

Process of collecting, organizing, storing, and representing data in a structured format so that it can be processed efficiently

```
class Employee
{
    int id;
    String name;
    double salary;
}
```

```
int[] marks = {85, 90, 78, 95};
```

Solution Evaluation

Solution Evaluation means **testing and verifying whether a solution works correctly for different inputs and produces the expected output**

```
int number = 10;

if(number % 2 == 0)
{
   System.out.println("Even");
}
else
{
   System.out.println("Odd");
}
```

Computational Thinking

Solution Evaluation and Optimization

Input	Expected Output	Actual Output	Result
10	Even	Even	Pass
15	Odd	Odd	Pass
0	Even	Even	Pass
-8	Even	Even	Pass
-7	Odd	Odd	Pass

Computational Thinking

Optimization

Optimization means achieving the same result with less time, less memory, or less effort.

You have two routes to your office.

- **Route A → 20 km,**
- **Route B → 15km**

Problem Generalization and Reusability

Creating a solution that works for many similar problems rather than just one specific case.

```
public  static int add(int a, int b)
{
   return a + b;
}
```

```
System.out.println(add(10,20));
System.out.println(add(50,100));
System.out.println(add(250,300));
```

Clean code principles

Clean Code is code that is **easy to read, understand, maintain, test, and modify**. It should be written not only for the computer to execute but also for other developers (and your future self) to understand.

Naming conventions

Code formatting

Write Clean functionality decomposed code

Clean code principles

Parameter and return value design

```
public static int add(int a, int b)
{
   return a + b;
}
```

```
public static void calculateSalary(int salary,
                                   String name,
                                   int age,
                                   String address)
{
   // Uses only salary
}
```

Write Clean functionality decomposed code

Clean code principles

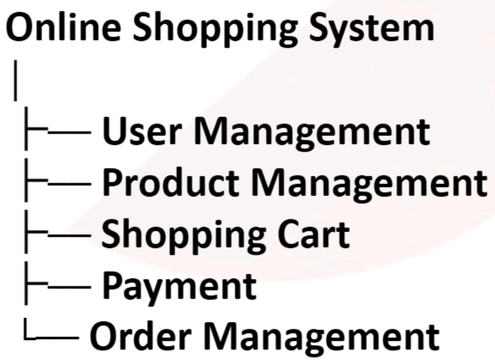
```
public static void add(int a, int b)
{
   System.out.println(a + b);
}
```

Write Clean functionality decomposed code

Clean code principles

Step 1: Decomposition (Problem Analysis)

You first analyze the problem and divide it into smaller parts.



Write Clean functionality decomposed code

Clean code principles

Modular Programming (Implementation)

Now you implement each part as a separate module (class or package)

UserModule.java
ProductModule.java
CartModule.java
PaymentModule.java
OrderModule.java

Write Clean functionality decomposed code

Clean code principles

Reusable Code Development (DRY Principle)

What is DRY?

DRY stands for **Don't Repeat Yourself**.

Write Clean functionality decomposed code

Clean code principles

```
System.out.println(10 + 20);

System.out.println(30 + 40);

System.out.println(100 + 200);
```

```
public static int add(int a, int b)
{
    return a + b;
}


System.out.println(add(10, 20));
System.out.println(add(30, 40));
System.out.println(add(100, 200));
```

Write Clean functionality decomposed code

Documentation and commenting

Documentation and Commenting is the practice of providing clear explanations about the software and the code to make it easier to understand, use, maintain, and modify.
Documentation explains the overall software, design, usage, or APIs.
Comments explain specific parts of the code when necessary.

Write Clean functionality decomposed code

Error handling technique

Normal Situation

- Insert card.
- Enter PIN.
- Withdraw cash.

Error Situations

- Incorrect PIN → Display "Invalid PIN."
- Insufficient Balance → Display "Insufficient Balance."
- ATM Out of Cash → Display "Service Unavailable."

Instead of stopping unexpectedly, the ATM informs the user about the problem and handles it appropriately.

Write Clean functionality decomposed code

Debugging

Debugging is a step-by-step approach used to identify, analyze, and fix errors in a program to ensure it runs correctly.

Write Clean functionality decomposed code

Debugging

Debugging is a step-by-step approach used to identify, analyze, and fix errors in a program to ensure it runs correctly.

Write Clean functionality decomposed code

Unit Testing

Unit Testing is the process of **testing an individual unit (such as a method, function, or class) independently** to verify that it works correctly before integrating it with other parts of the application.

```
public static int add(int a, int b) { return a + b; }
```

Write Clean functionality decomposed code

Input	Expected Output	Result
add(10,20)	30	Pass
add(5,8)	13	Pass
add(-2,5)	3	Pass
add(0,0)	0	Pass

Write Clean functionality decomposed code

```
import static org.junit.jupiter.api.Assertions.assertEquals;
import org.junit.jupiter.api.Test;

class CalculatorTest
{
    @Test
    void testAdd()
    {
        assertEquals(30, Calculator.add(10, 20));
    }
}
```

JUnit automatically checks whether the actual result matches the expected result.

THANK YOU